

Sheet3__Neural Network

March 10, 2023

Name	ID	Department
Abdelrhman Mohamed Abdelhady Hodib	2022513643	Intelligent system

1 Questions

1.1 What is a perceptron?

A perceptron is a simple neural network model that is used for binary classification problems

1.2 List the activation function used in perceptron and the differences between them?

The activation function used in perceptron is usually a step function or a sigmoid function. A step function outputs either 0 or 1, while a sigmoid function outputs a continuous value between 0 and 1

1.3 How is the output of a perceptron computed?

By taking the sum of the weighted input values and passing it through an activation function

1.4 What is the role of a loss function in the training of a perceptron?

The role of a loss function is to measure the difference between the predicted output and the actual output for an input.

1.4.1 Define mean squared error (MSE) loss function.

used to calculate the average squared difference between the predicted output and the actual output for an input.

$$\text{MSE} = (1/n) * \sum (y - \hat{y})^2$$

1.5 How can you adjust the weights of a perceptron during training?

The gradient descent approach can be used to modify a perceptron's weights as it is being trained. In order to minimize the loss function, the method calculates the gradient of the loss function with respect to the weights and then updates the weights in the opposite direction as the gradient.

1.6 Why do we apply `sign()` function on the perceptron output?

The perceptron's continuous output is changed into a binary output of either 0 or 1 using the `sign()` function.

1.7 What is the use of Bias?

Bias is used to shift the activation function to the left or right

1.8 How to update Bias?

The gradient of the loss function with respect to the bias can be calculated, and the bias can then be changed in the same manner as the weights.

1.9 Design one layer perceptron to represent the following logic functions:

- AND • OR • NOT

AND:

Input 1	Input 2	Output
0	0	0
0	1	0
1	0	0
1	1	1

OR:

Input 1	Input 2	Output
0	0	0
0	1	1
1	0	1
1	1	1

NOT:

0 will be 1

1 will be 0

1.10 Given the table below, Consider a perceptron with weights $w_1 = -0.5$, $w_2 = 0.1$, and $w_3 = 0.4$, bias $b = 0.2$, and learning rate $= 0.01$. Train for 2 iteration with weights updated after each sample and then calculate the accuracy

```
[2]: import numpy as np

# step function
def step(x):
```

```

    return np.where(x >= 0, 1, -1)

# weights and bias
w = np.array([-0.5, 0.1, 0.4])
b = 0.2

lr = 0.01
n_iter = 2

# the dataset
X = np.array([[0.2, 0.5, 0.7],
               [0.1, 0.3, 0.6],
               [0.4, 0.8, 0.2],
               [0.6, 0.4, 0.9]])
y = np.array([1, -1, 1, -1])

for i in range(n_iter):
    for j in range(len(X)):

        z = np.dot(X[j], w) + b
        y_pred = step(z)

        error = y[j] - y_pred

        w += lr * error * X[j]
        b += lr * error

y_pred = step(np.dot(X, w) + b)
accuracy = np.mean(y_pred == y)
print("Accuracy:", accuracy)
print("Weights:", w)
print("Bias:", b)

```

```

Accuracy: 0.5
Weights: [-0.528  0.072  0.34 ]
Bias: 0.12000000000000004

```

2 Coding Question

2.1 Write code to repeat question 11 but with the weights updated after a full iteration using MSE loss, compare the final weights and accuracy.

```

[21]: # Define hyperparameters
learning_rate = 0.01
epochs = 2

# Define Mean Squared Error (MSE) loss function

```

```

def mse_loss(y_true, y_pred):
    return np.mean(np.square(y_true - y_pred))

for epoch in range(epochs):
    shuffle_indices = np.random.permutation(len(X))
    X = X[shuffle_indices]
    y = y[shuffle_indices]

    dw = np.zeros_like(w)
    db = 0

    for i in range(len(X)):
        z = np.dot(w, X[i]) + b
        y_pred = np.sign(z)

        error = y[i] - y_pred

        dw += learning_rate * error * X[i]
        db += learning_rate * error

    w += dw
    b += db

    y_pred = np.sign(np.dot(X, w) + b)
    accuracy = np.mean(y_pred == y)

    print(f"Epoch {epoch+1} -- Loss: {mse_loss(y, y_pred):.4f} -- Accuracy: ␣
↪{accuracy:.4f}")

print(f"\nFinal weights: {w}")
print(f"Final bias: {b}")

```

Epoch 1 -- Loss: 1.0000 -- Accuracy: 0.7500

Epoch 2 -- Loss: 0.0000 -- Accuracy: 1.0000

Final weights: [-0.41 0.29 0.002]

Final bias: -0.05999999999999997